WEEK-2 PRACTICAL

Task 1: Develop a C program that

demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID CODE:

```
 GNU nano 8.7.1                          week2.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;
    char command[100];

    printf("Enter Linux command: ");
    scanf("%s", command);

    pid = fork();

    if(pid == 0)
    {
        printf("Child Process ID: %d\n", getpid());
        execlp(command, command, "-a", NULL);
    }
    else
    {
        wait(NULL);
        printf("Parent Process ID: %d\n", getpid());
        printf("Child process completed.\n");
    }

    return 0;
}
```

Child process completed.

OUTPUT:

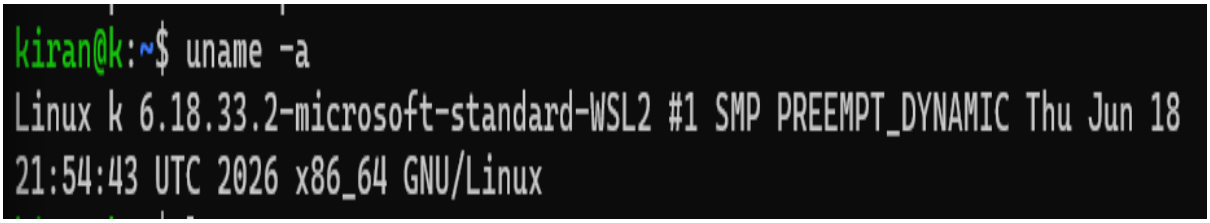
```
kiran@k:~$ nano week2.c
kiran@k:~$ gcc week2.c -o week2
kiran@k:~$ ./week2
Enter Linux command: uname
Child Process ID: 2089
Linux k 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 18
21:54:43 UTC 2026 x86_64 GNU/Linux
Parent Process ID: 2088
Child process completed.
```

Child process completed.

REPORT: I executed the C program successfully. The program accepted a Linux command from the user, created a child process using for The uname -a command displayed complete information about my Linux system, including the kernel version, system architecture, hostname, and operating system details.k(), and executed the command using the exec() system call. The parent process waited until the child process finished execution. Finally, the program displayed the process IDs of both the parent and child processes, and the message "Child process completed." was displayed.

Task-2: Using Linux terminal commands (uname, lscpu, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

Uname-a:



REPORT: The uname -a command displayed complete information about my Linux system, including the kernel version, system architecture, hostname, and operating system details.

Lscpu:



```
kiran@k:~$ lscpu
Architecture:                       x86_64
  CPU op-mode(s):                   32-bit, 64-bit
  Address sizes:                    48 bits physical, 48 bits virtual
  Byte Order:                       Little Endian
CPU(s):                             12
  On-line CPU(s) list:              0-11
Vendor ID:                          AuthenticAMD
  Model name:                       AMD Ryzen 5 7530U with Radeon Graphics
    CPU family:                     25
    Model:                          80
    Thread(s) per core:             2
    Core(s) per socket:             6
    Socket(s):                      1
    Stepping:                       0
    BogoMIPS:                       3992.40
    Flags:                          fpu vme de pse tsc msr pae mce cx8 apic sep mtr
                                    r pge mca cmov pat pse36 clflush mmx fxsr sse s
                                    se2 ht syscall nx mmxext fxsr_opt pdpelgb rdtsc
                                    p lm constant_tsc rep_good nopl xtopology tsc_r
                                    eliable nonstop_tsc cpuid extd_apicid tsc_known
                                    _freq pni pclmulqdq ssse3 fma cx16 sse4_1 sse4_
                                    2 movbe popcnt aes xsave avx f16c rdrand hyperv
                                    isor lahf_lm cmp_legacy svm cr8_legacy abm sse4
                                    a misalignsse 3dnowprefetch osvw topoext perfct
                                    r_core ssbd ibrs ibpb stibp vmmcall fsgsbase bm
                                    i1 avx2 smep bmi2 erms invpcid rdseed adx smap
                                    clflushopt clwb sha_ni xsaveopt xsavec xgetbv1
                                    xsaves clzero xsaveerptr arat npt nrip_save tsc
                                    _scale vmcb_clean flushbyasid decodeassists pau
                                    sefilter pfthreshold v_vmsave_vmload umip vaes
                                    vpclmulqdq rdpid fsrm
Virtualization features:
  Virtualization:                   AMD-V
  Hypervisor vendor:                Microsoft
  Virtualization type:              full
Caches (sum of all):
  L1d:                              192 KiB (6 instances)
  L1i:                              192 KiB (6 instances)
  L2:                               3 MiB (6 instances)
  L3:                               16 MiB (1 instance)
NUMA:
  NUMA node(s):                     1
  NUMA node0 CPU(s):                0-11
Vulnerabilities:
  Gather data sampling:             Not affected
  Ghostwrite:                       Not affected
  Indirect target selection:        Not affected
  Itlb multihit:                    Not affected
  L1tf:                             Not affected
  Mds:                              Not affected
  Meltdown:                         Not affected
  Mmio stale data:                  Not affected
  Old microcode:                    Not affected
  Reg file data sampling:           Not affected
  Retbleed:                         Not affected
  Spec rstack overflow:             Vulnerable: Safe RET, no microcode
  Spec store bypass:                Mitigation; Speculative Store Bypass disabled v
                                    ia prctl
  Spectre v1:                       Mitigation; usercopy/swapgs barriers and __user
                                     pointer sanitization
  Spectre v2:                       Mitigation; Retpolines; IBPB conditional; IBRS_
                                    FW; STIBP always-on; RSB filling; PBRSB-eIBRS N
                                    ot affected; BHI Not affected
  Srbds:                            Not affected
  Tsa:                              Mitigation; Clear CPU buffers
  Tsx async abort:                  Not affected
  Vmscape:                          Not affected
```

REPORT: The lscpu command displayed the CPU information of my system, such as the processor architecture, number of CPU cores, CPU model, and other hardware details. This helped me understand the processor resources available.

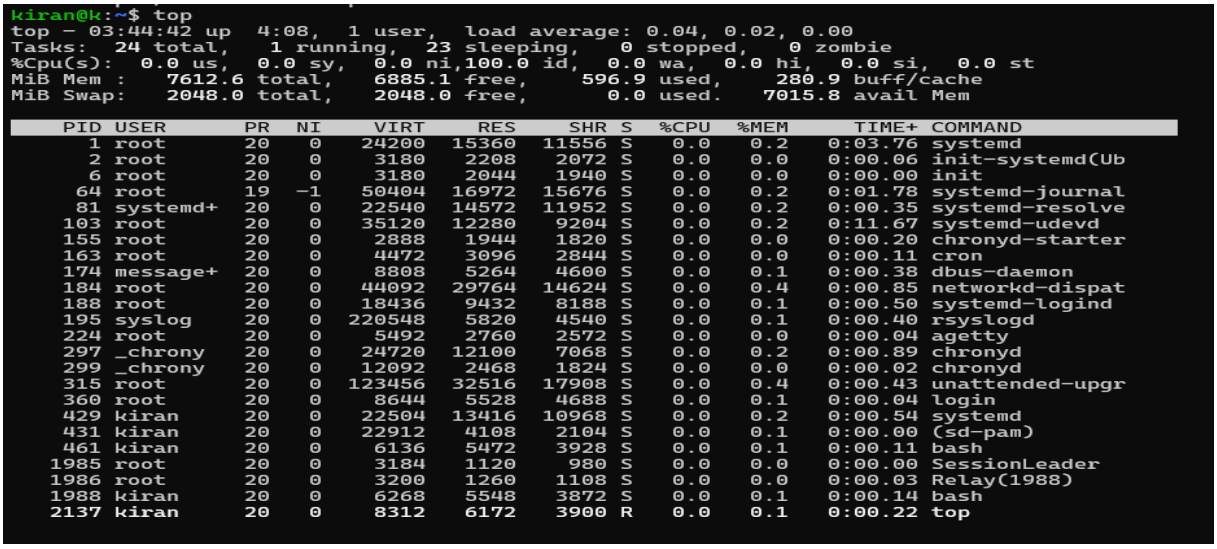
Ps:



```
kiran@k:~$ ps
    PID TTY          TIME CMD
   1988 pts/0    00:00:00 bash
   2121 pts/0    00:00:00 ps
```

REPORT: The ps command displayed the processes currently running in my terminal. I observed the process IDs and command names of the active processes.

Top:



REPORT: The top command displayed the live status of the system. I observed the CPU usage, memory usage, running processes, and system performance updating continuously.